SMARTBEAR
BugSnag

# 2026 BugSnag Application Stability Index

Stability benchmarks that empower you to make informed decisions about the impact of bugs in production

SMARTBEAR

SMARTBEAR
BugSnag

2026

# BugSnag Application Stability Index

Stability benchmarks that empower you to make informed decisions about the impact of bugs in production

IN THE REPORT

Will update design once a cover is chosen

# Introduction

**Development teams constantly balance building new features with fixing existing bugs. Figuring out what to prioritize often comes down to the question, "Is this application stable enough?" But without benchmarks, coming up with an answer is difficult.**

The 2026 BugSnag Application Stability Index from SmartBear analyzes 241 applications handling more than 25 million sessions monthly, providing stability and release frequency benchmarks across industries, platforms, and devices. These figures help developers answer questions like:

- "BugSnag shows our application has a 99.87% stability score. Is that good?"
- "How do we compare to others in our industry?"
- "Can we release more frequently? When should we prioritize fixing bugs?"

## Our Analysis

**Stability depends on practices, not just cadence**

Throughout this paper, you'll see variation across industries for both number of deployments per month and stability. Interestingly, you'll find examples where stability is nearly identical independent of release frequency. High stability comes from strong practices, not release frequency.

**Small percentage point gaps create massive user impact**

An app with 747 million monthly sessions at 99.69% stability experiences 2.3 million crashed booking sessions per month, each one a potential lost transaction or dissatisfied customer. Inching up the stability to 99.95% would reduce crashes to just 374,000. That 0.26 percentage point difference means 6 times more failures.

**Industry benchmarks make stability targets actionable.**

Benchmarking can help development teams determine what to prioritize. For example, e-commerce apps achieved 99.95% median stability. An e-commerce company many decide to set a stability goal of 99.90% (close to the median with some breathing room) and a critical service level agreement of 99.85% (the "stop everything and fix bugs" threshold). These data-backed benchmarks replace guesswork with evidence.

# Methodology

**We calculated stability scores using data from BugSnag's SaaS deployment, based on crash-free session rates and error-free user rates over 30 days.**

Industry and platform benchmarks are medians across all qualifying applications, not weighted by traffic volume. Release frequency reflects the median number of production releases for each application.

## How We Measure Application Stability

BugSnag's stability score tracks how often applications work without crashes or critical errors. It's shown as a percentage, similar to how infrastructure teams measure uptime with "five nines" (99.999%). Higher stability scores correlate with healthier applications that don't fail during critical workflows (like the checkout process, for instance). When the stability score is high, developers work on building. When it drops below a certain threshold, they shift to debugging.

### Stability can be calculated in two ways

**Crash-free session rate**
The percentage of user sessions that are completed without an unhandled exception or crash. A session begins when a person opens an application and ends either when they close it or the app crashes.

**Error-free user rate**
The percentage of daily active users who don't experience errors when using the application. This user-centric view helps teams understand how many people have flawless experiences.

These measurements provide a complete picture of application stability from a technical perspective (how often does it perform without crashing) and a user perspective (how many people have good experiences). The benchmarks in this paper use whichever metric the organizations chose to monitor.

$$\text{Stability Score} = \left(1 - \frac{\text{crashed user sessions}}{\text{total user sessions}}\right) \times 100$$

# How to Use the Report's Benchmarks

This report provides two types of benchmarks to help you set appropriate goals and understand your application's performance: stability scores and release frequency distributions.

## Using Stability Benchmarks In Your Organization

Suppose the median stability score for your industry is 99.90%.
You might set the following goals for your organization:

**Target stability (Service Level Agreement "SLA")**

**99.85%**

A competitive score close to the median while allowing for flexibility if stability slips occasionally.

**Critical stability (Service Level Object "SLO"):**

**99.80%**

Your absolute minimum stability score before triggering crisis mode.

### These thresholds give development leaders guidance on prioritization

**99.90%**

ABOVE TARGET

Continue developing features.

**99.83%**

BELOW TARGET BUT ABOVE CRITICAL

Increase focus on fixing bugs.

**99.81%**

BELOW CRITICAL

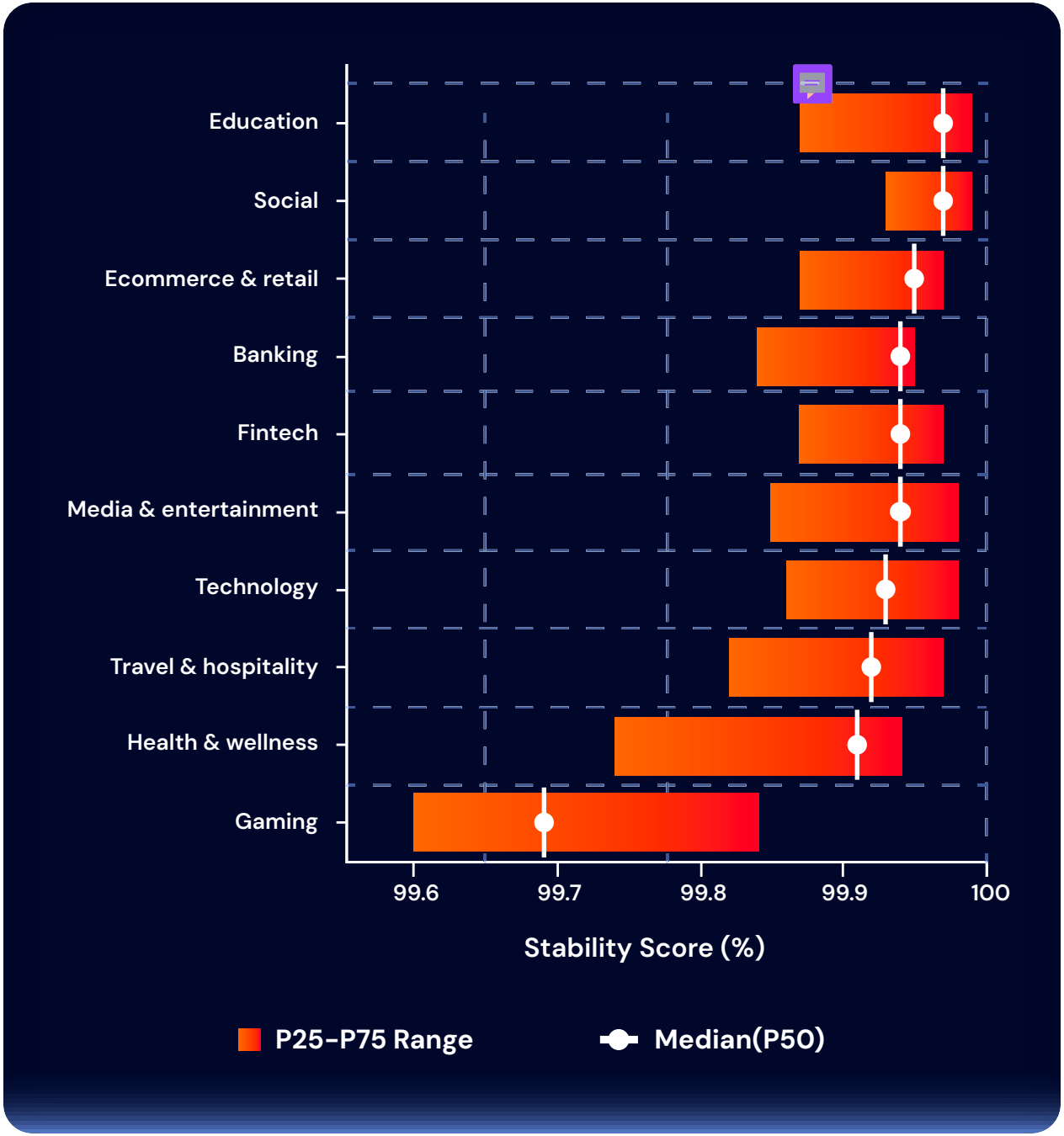
Stop building features, prioritize stability debugging.

# Measuring Stability Across Industries

**Application stability improved across most industries from 2022 (when we last published the report) to 2025, with several sectors achieving notable gains.**

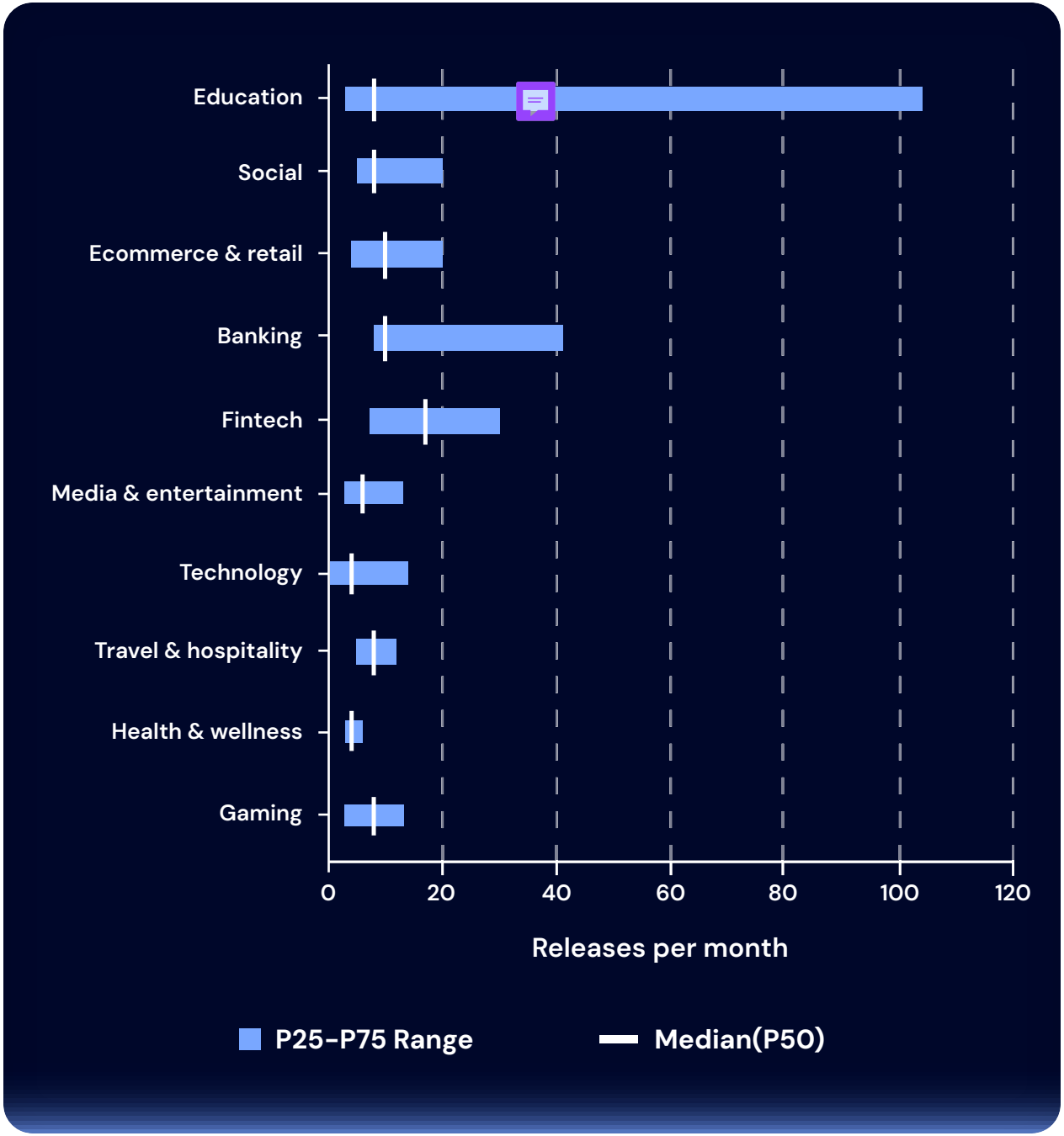
Education rose from 99.93% to 99.97%, now leading all industries. Health & wellness (applications include ones for health IT, clinic management, and on-demand workout classes) showed significant progress, jumping from 99.71% to 99.91%. Gaming made the most dramatic improvement, rising from 99.60% to 99.69%, but is still the lowest score.

While the gap between education and gaming may seem small (0.28 percentage points), it can have a big impact at scale. A gaming app with 100 million monthly sessions at 99.69% stability experiences 310,000 sessions crash. An education app with the same traffic sees only 30,000 crashes. That's 10 times fewer failures from less than a third of a percentage point. This shows why choosing your thresholds carefully is important.



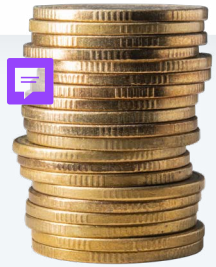
# Release Frequency by Industry: Balancing Speed and Stability

Application release frequency varies across industries, reflecting different approaches to balancing feature building with stability. Understanding these patterns helps teams benchmark deployment frequency and identify opportunities to accelerate or stabilize release cycles.



## Release Frequency and Stability: Initial Observations

Our data shows varied patterns across industries when comparing release frequency to stability scores:



### FINANCE & BANKING

**10**
releases/month

**99.94%**
stability


### FINTECH

**17**
releases/month

**99.94%**
stability

### TECHNOLOGY

**4**
releases/month

**99.93%**
stability

### MEDIA & ENTERTAINMENT

**6**
releases/month

**99.94%**
stability

Since 2022, we've added a category for Fintech, separating it from Financial Services, due to its release cadence differing significantly. Stability remains similar between the two.

Frequent deployments don't necessarily equal greater stability. High-performing teams maintain excellent stability whether they deploy 6 times or 48 times per month. The difference lies in deployment practices, testing rigor, error monitoring, and organizational maturity rather than cadence alone.

Teams should focus on stability practices that work for their release frequency rather than assuming slower or faster deployments will inherently improve reliability.

# Comparing Web and Mobile Application Stability

**Web applications achieve 99.94% median stability across our dataset, edging out mobile apps at 99.92%.**

While this 0.02-percentage-point gap appears tiny, it means a different user experience at scale. At 100 million monthly sessions, this gap translates to 20,000 additional crashes for mobile users compared to web users.

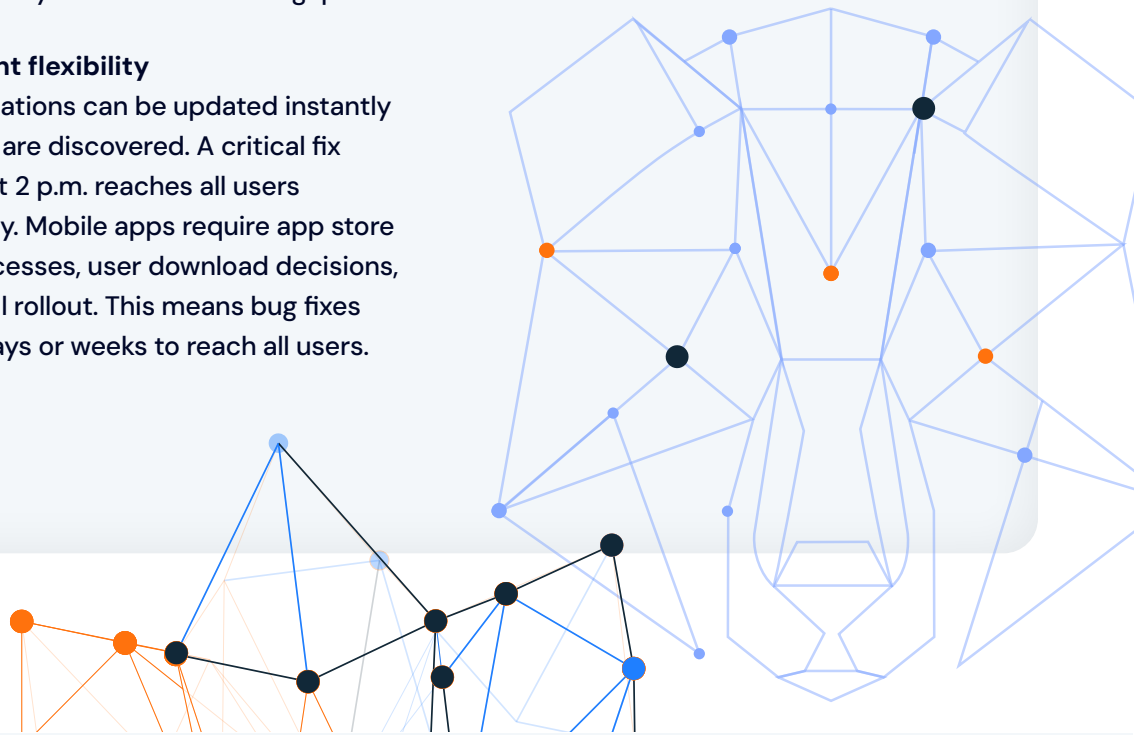
## Why Web Maintains an slight Advantage

Several factors contribute to web applications' stability edge:

**Controlled environment**
Web applications run in browsers where developers can test against a limited set of rendering engines (Chrome, Safari, Firefox, Edge). Mobile developers face a fragmented landscape. There are dozens of device manufacturers, OS versions, and hardware configurations. While we can't determine exact causes from stability data alone, mobile's additional complexity from things like device fragmentation and OS variations likely contributes to this gap.

**Deployment flexibility**
Web applications can be updated instantly when bugs are discovered. A critical fix deployed at 2 p.m. reaches all users immediately. Mobile apps require app store review processes, user download decisions, and gradual rollout. This means bug fixes can take days or weeks to reach all users.

**Testing complexity**
Web developers can reasonably test across major browsers and screen sizes. Mobile developers face great complexity: iOS 15, 16, 17, 18 across numerous iPhone models, and an even more complex fragmented Android landscape. Comprehensive mobile testing requires significantly more resources and infrastructure.

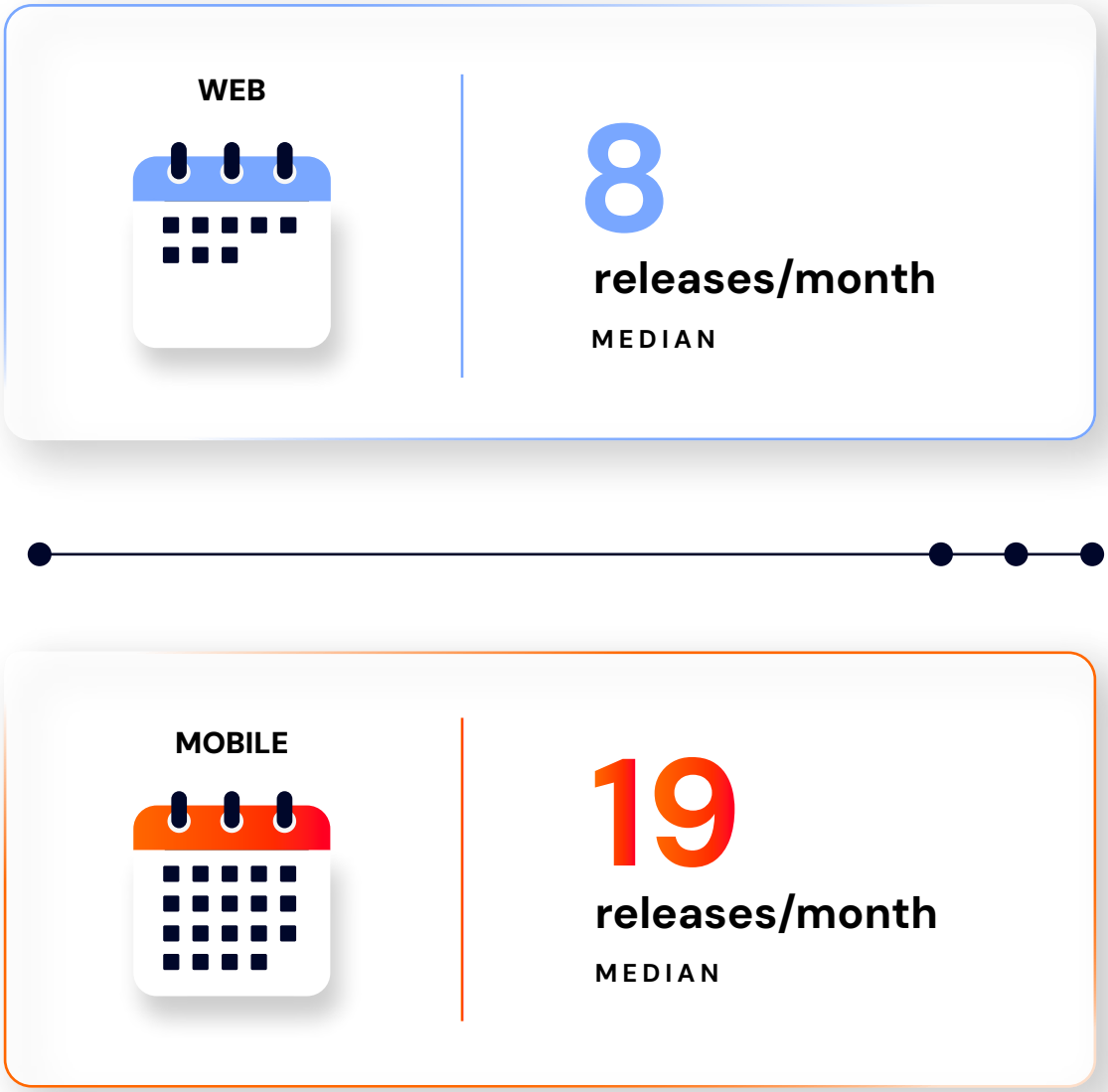


# Web vs. Mobile: The Release Frequency Gap

**Contrary to conventional wisdom that web applications release more frequently due to instant deployment capabilities, our data reveals mobile applications actually release more than twice as frequently: 19 releases per month compared to web's 8 releases per month.**

This finding challenges common assumptions about platform deployment dynamics. While web applications can technically deploy instantly without app store approval, mobile teams have developed practices that result in higher release frequencies despite facing app store review processes.

This is notable when we consider that the above section showed how close mobile and web stability is: More than doubling the number of releases for mobile had little to no impact on stability.



## Why Mobile Releases More Frequently Despite Technical Constraints

Several factors enable mobile's faster release cadence even with app store gatekeeping:

**App store dynamics**
App store ratings and reviews directly impact discoverability and downloads. A single visible bug can trigger negative reviews that damage reputation and deter potential users. This creates business pressure to ship fixes rapidly, even when it means navigating app store review processes.

**Platform fragmentation**
Mobile developers must support numerous device types, OS versions, and hardware configurations simultaneously. This complexity requires rapid iteration to address device-specific issues that only surface in production across diverse user environments.

Working on this page design. Any suggestion for a pull quote or data callout?

**User expectations**
Mobile ecosystems have conditioned users to expect frequent updates with incremental improvements and bug fixes. Apps that don't maintain regular release cadences risk users switching to competitors, directly impacting retention and revenue.

**Feature experimentation**
Mobile apps commonly use progressive delivery techniques (feature flags, A/B tests, and phased rollouts) to test changes with user segments before full deployment. These controlled rollouts often register as distinct releases even though features aren't universally available, inflating release counts compared to web's all-or-nothing deployments.

## Why Web Releases Less Frequently Despite Technical Advantages

Web applications' 8 releases per month seems paradoxical given their deployment advantages but reflects deliberate strategic choices.

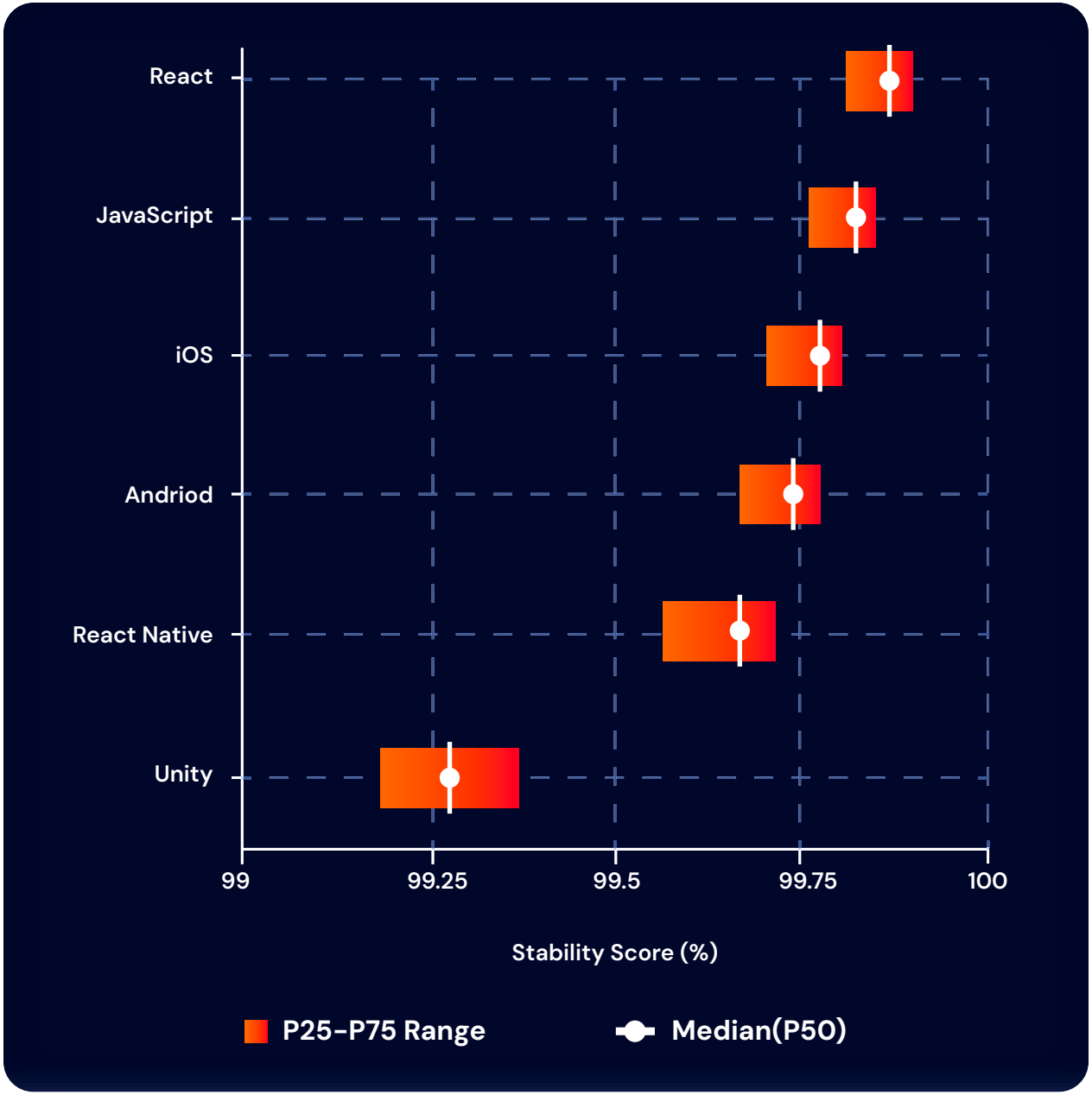
**Bundling over continuous deployment**
Despite the technical ability to deploy updates instantly, web teams often bundle multiple fixes and features into scheduled releases rather than deploying changes as soon as they're ready. This creates more substantial updates released less frequently.

**Reduced deployment urgency**
Web deployments reach all users immediately without requiring app store approval or user-initiated updates. This eliminates the forcing function that drives mobile's release frequency. Teams can afford to wait and bundle changes without worrying about update adoption rates.

# Platform Performance: Stability Across Development Frameworks

**Application stability varies across development frameworks, reflecting different technical challenges, error types, and developer ecosystems.**

The 0.29 percentage point spread between React (99.98%) and Unity (99.69%) represents the full range of platform performance, with most platforms having a median stability score between 99.88% and 99.98%.



## Web Platforms: Modern Framework Consistency

React (99.98%) and JavaScript (99.91%) demonstrate the maturity of modern web development frameworks. Both platforms benefit from sophisticated error handling capabilities built into contemporary JavaScript frameworks and robust browser developer tools. Web applications measure stability across multiple browsers (Chrome, Firefox, Safari) meaning a single code error can impact users differently depending on their browser. Despite this complexity, web platforms maintain consistently high stability scores.

React's slight edge over JavaScript (0.07 percentage points) may reflect React's architectural advantages, including Error Boundaries that prevent component-level errors from cascading into full application crashes. However, both platforms achieve stability above 99.90%, indicating that modern web frameworks provide strong foundations for reliable applications.

## Native Mobile Platforms: Specialized Development Ecosystems

iOS (99.95%) and Android (99.93%) both achieve stability above 99.90%, reflecting mature development ecosystems with deep specialist knowledge.

iOS's marginally higher stability (0.02 percentage points) occurs despite both platforms having mature developer communities. Android's development environment supports a significantly wider range of devices (different manufacturers, hardware capabilities, and OS versions) creating more variables for developers to manage. iOS development targets a smaller and more standardized set of devices, reducing platform fragmentation compared to Android.

## Cross-Platform Mobile: Trading Specialization for Efficiency

React Native (99.88%) enables simultaneous Android and iOS development using JavaScript and React patterns. This cross-platform approach attracts developers with web experience who may not specialize in native mobile frameworks, allowing teams to maintain both platforms with shared codebases.

React Native's 99.88% stability sits 0.07 percentage points below iOS and 0.05 percentage points below Android. This modest gap suggests that cross-platform frameworks can approach native platform stability while offering development efficiency gains.

React Native applications face error types from both worlds: JavaScript runtime errors common in web applications plus mobile-specific crashes and ANRs. Managing this dual complexity while maintaining 99.88% stability demonstrates the framework's maturity.

### Stability by Platform

Web
**99.98%**

iOS
**99.95%**

Android
**99.93%**

React Native
**99.88%**

## Unity: Gaming's Unique Challenges

Unity (99.69%) shows notably lower stability than other platforms. Unity is predominantly used in mobile gaming, which coincides with lower median stability in this dataset.

Gaming applications introduce technical complexity beyond typical mobile apps, including rapid content updates (new characters, levels, gameplay mechanics), asset loading and memory management for graphics-heavy content, and multiplayer synchronization and network state management.

Games also operate under different feature velocity pressures. Users expect frequent content updates, creating pressure to ship updates quickly. This rapid iteration may cause higher error rates as a trade-off for meeting customer needs.

Unity's 99.69% stability, while the lowest among frameworks analyzed, means fewer than 1 in 300 sessions encounter stability issues. For many gaming applications, this represents an acceptable balance between feature velocity and reliability.

# Conclusion

Stability is not just a technical metric, it's a business outcome. The data in this report shows how development teams can stop relying on instinct or internal debate and use data to decide when to ship a new feature and when to fix a bug. With clear benchmarks grounded in real-world application performance, teams can set meaningful stability targets, define actionable thresholds, and make prioritization decisions with confidence.